Machine Learning Practical Exam: Viva Prep Guide

Oct 27, 2026 · @Wint

Start here: how to use this guide

You will get one random practical out of 11, so learn the shared pattern first and then read your practical's section. Every practical follows the same flow: load a dataset, split train/test, fit a model, evaluate, plot, explain the graph.

Tonight's plan (about 3 hours):

1. Read this section and the cheat sheet at the end (30 min).
2. For each practical, read Aim, Steps and the graph explanation (10 min each).
3. Skim the viva Q&A for each; say the answers aloud once.
4. Run at least 3 codes yourself in Colab or Jupyter (1, 3, 8 are the quickest).

Standard imports (use in almost every practical):

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn import datasets
```

Built-in open-access datasets you can load without downloading anything:

Dataset	Load with	Type	Best for practical
Iris	`datasets.load_iris()`	150 rows, 4 features, 3 classes	2, 7, 8, 9, 10
Wine	`datasets.load_wine()`	178 rows, 13 features, 3 classes	2, 5, 7, 10

Dataset	Load with	Type	Best for practical
Breast Cancer	`datasets.load_breast_cancer()`	569 rows, 30 features, 2 classes	2, 6, 7, 10
Diabetes	`datasets.load_diabetes()`	442 rows, 10 features, numeric target	3, 4, 5
California Housing	`datasets.fetch_california_housing()`	20640 rows, 8 features, numeric target	3
Titanic (seaborn)	`sns.load_dataset('titanic')`	has missing values and categories	1, 6
Digits	`datasets.load_digits()`	1797 rows, 64 features	2

General viva questions (asked in any practical)

Q1. What is machine learning? A: Machine learning is a field where computers learn patterns from data and improve at a task without being explicitly programmed with rules for it.

Q2. Types of machine learning? A: Supervised (labelled data: regression, classification), unsupervised (no labels: clustering, dimensionality reduction) and reinforcement learning (an agent learns from rewards by interacting with an environment).

Q3. Difference between regression and classification? A: Regression predicts a continuous number (house price). Classification predicts a category (spam or not spam).

Q4. Why do we split data into train and test sets? A: To check how well the model works on unseen data. Testing on training data would give an over-optimistic score and hide overfitting. Typical split is 80/20 or 70/30.

Q5. What is `random_state` in `train_test_split`? A: A seed that makes the random split reproducible, so you get the same split every run.

Q6. What is overfitting and underfitting? A: Overfitting: the model memorises the training data, so training score is high but test score is low (high variance). Underfitting: the model is too simple to capture the pattern, so both scores are low (high bias).

Q7. What is cross-validation? A: The data is split into k folds. The model trains on k-1 folds and tests on the remaining one, repeated k times, and the scores are averaged. It gives a more reliable estimate than a single split.

Q8. Why do you fit the scaler only on training data? A: Fitting on the full data leaks test-set information (mean, variance) into training, called data leakage. Use `fit_transform` on train and only `transform` on test.

Q9. What is a feature and what is a label/target? A: Features (X) are the input variables. The label or target (y) is the output the model must predict.

Q10. What does `fit()`, `predict()` and `score()` do in scikit-learn? A: `fit` trains the model, `predict` gives outputs for new data, `score` returns the default metric (R^2 for regressors, accuracy for classifiers).

Q11. Why did you choose this dataset? A: Say it is open access (built into scikit-learn or UCI/Kaggle), small enough to run fast, and suits the algorithm (for example, Iris has well-separated classes, good for clustering and SVM).

Q12. What is a hyperparameter? How is it different from a parameter? A: Parameters are learned from data (regression coefficients). Hyperparameters are set before training (k in K-Means, C in SVM, alpha in Ridge, learning rate).

Q13. What is a pipeline? A: `sklearn.pipeline.Pipeline` chains steps (imputer, scaler, model) so they run in order and avoid leakage.

Practical 1: Feature engineering (missing values, encoding, scaling)

Aim: Prepare a clean, model-ready dataset by handling missing values, encoding categorical variables and scaling numerical attributes.

Dataset: Titanic (`sns.load_dataset('titanic')`), because it has missing values in `age` and `deck`, and categorical columns like `sex` and `embarked`.

Steps

1. Load the data and inspect it with `df.info()`, `df.describe()` and `df.isnull().sum()`.
2. Handle missing values: fill numeric columns with the median, categorical columns with the mode, drop columns with too many missing values (`deck`).
3. Encode categorical variables: label encoding for ordered/binary columns, one-hot encoding for nominal columns.
4. Scale numeric columns with StandardScaler (mean 0, std 1) or MinMaxScaler (range 0 to 1).
5. Print the cleaned dataset's head and confirm no nulls remain.

Code

```
import pandas as pd, seaborn as sns
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder

df = sns.load_dataset('titanic')
print(df.isnull().sum())

# 1. Missing values
df = df.drop(columns=['deck'])                        # ~77% missing
df['age'] = df['age'].fillna(df['age'].median())
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])
df['embark_town'] = df['embark_town'].fillna(df['embark_town'].mode()[0])
print(df.isnull().sum().sum())                        # should be 0

# 2. Encoding
df['sex'] = LabelEncoder().fit_transform(df['sex'])  # male=1, female=0
df = pd.get_dummies(df, columns=['embarked', 'class'], drop_first=True)

# 3. Scaling
num_cols = ['age', 'fare']
df[num_cols] = StandardScaler().fit_transform(df[num_cols])
print(df.head())
```

Output to explain: After cleaning, the null count is 0, categorical text became numbers (0/1 columns), and `age` and `fare` have mean about 0 and std about 1.

Viva questions and answers

Q1. What is feature engineering? A: Creating, transforming and selecting input variables so that the model can learn better. It includes cleaning, encoding, scaling, creating new features and selecting useful ones.

Q2. Why do we handle missing values? A: Most algorithms cannot work with NaN and raise errors. Ignoring them can also bias results.

Q3. What are the ways to handle missing values? A: Delete rows or columns, fill with mean/median/mode, fill with a constant, forward/backward fill, or model-based imputation (KNN imputer).

Q4. When do you use mean, median and mode? A: Mean for symmetric numeric data, median for skewed data or data with outliers (age, income), mode for categorical data.

Q5. Why did you drop the `deck` column? A: About 77% of its values are missing. Filling that many values would invent information, so dropping it is safer.

Q6. What is the difference between label encoding and one-hot encoding? A: Label encoding assigns an integer to each category (0, 1, 2) and suits ordinal data. One-hot creates a separate 0/1 column per category and suits nominal data, because it does not imply any order.

Q7. What is the dummy variable trap? A: If you keep all one-hot columns, one column is a perfect combination of the others (multicollinearity). `drop_first=True` removes one column to avoid this.

Q8. Why is scaling needed? A: Features with large ranges (fare in hundreds) can dominate features with small ranges (age). Distance-based and gradient-based algorithms (KNN, SVM, K-Means, PCA, logistic regression, neural networks) need scaled data.

Q9. Difference between standardisation and normalisation? A: Standardisation: z = (x - mean) / std, giving mean 0 and std 1. Normalisation (min-max): x′ = (x - min) / (max - min), giving values in [0, 1].

Q10. Which algorithms do not need scaling? A: Tree-based models: Decision Tree, Random Forest, Gradient Boosting.

Q11. What is an outlier and how does scaling treat it? A: An outlier is an extreme value far from the rest. Min-max scaling is very sensitive to outliers; StandardScaler is less so; RobustScaler uses median and IQR and is the most robust.

Q12. What is data leakage? A: When information from the test data influences training, for example fitting the scaler on the whole dataset before splitting. It gives falsely good results.

Q13. What is the difference between `fit`, `transform` and `fit_transform`? A: `fit` learns parameters (mean, std), `transform` applies them, `fit_transform` does both in one step.

Q14. What is a categorical vs numerical variable? A: Categorical holds labels or groups (sex, city). Numerical holds measurable numbers (age, fare). Numerical can be discrete (count) or continuous (height).

Q15. How do you detect missing values in pandas? A: `df.isnull().sum()` gives the count per column; `df.info()` shows non-null counts; a heatmap `sns.heatmap(df.isnull())` shows them visually.

Practical 2: Dimensionality reduction with PCA and LDA

Aim: Apply PCA and LDA on a high-dimensional dataset and observe how reducing features affects model performance and computational time.

Dataset: Breast Cancer (30 features) or Wine (13 features).

Steps

1. Load data, split into train/test, standardise the features.
2. Train a baseline classifier (Logistic Regression) on all features; record accuracy and training time.
3. Apply PCA (unsupervised) with n_components = 2 (or enough for 95% variance); retrain and compare.
4. Apply LDA (supervised) with n_components = min(classes - 1, features); retrain and compare.
5. Plot the explained variance ratio, and a 2D scatter of PCA and LDA outputs coloured by class.
6. Compare accuracy and time in a table.

Code

```
import time, numpy as np, matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
sc = StandardScaler(); Xtr = sc.fit_transform(Xtr); Xte = sc.transform(Xte)

def run(name, A, B):
    t = time.time()
    m = LogisticRegression(max_iter=1000).fit(A, ytr)
    print(name, 'acc =', round(accuracy_score(yte, m.predict(B)), 4), 'time
=', round(time.time()-t, 4))

run('Original (30 features)', Xtr, Xte)

pca = PCA(n_components=2).fit(Xtr)
Xtr_p, Xte_p = pca.transform(Xtr), pca.transform(Xte)
run('PCA (2 comps)', Xtr_p, Xte_p)
print('Explained variance ratio:', pca.explained_variance_ratio_)

lda = LDA(n_components=1).fit(Xtr, ytr)          # 2 classes -> max 1
```

```
component
Xtr_l, Xte_l = lda.transform(Xtr), lda.transform(Xte)
run('LDA (1 comp)', Xtr_l, Xte_l)

# Graph 1: cumulative explained variance
pca_full = PCA().fit(Xtr)
plt.plot(np.cumsum(pca_full.explained_variance_ratio_), marker='o')
plt.axhline(0.95, color='r', ls='--')
plt.xlabel('Number of components'); plt.ylabel('Cumulative explained
variance')
plt.show()

# Graph 2: PCA scatter
plt.scatter(Xtr_p[:,0], Xtr_p[:,1], c=ytr, cmap='coolwarm', alpha=0.7)
plt.xlabel('PC1'); plt.ylabel('PC2'); plt.show()
```

Typical result: 30 features give about 97 to 98% accuracy. PCA with 2 components gives about 94 to 96%. LDA with 1 component gives about 95 to 97%. Training time drops a little because there are fewer features.

Graph questions

G1. What does the cumulative explained variance plot show? A: The x-axis is the number of principal components, the y-axis is the share of total variance they capture together. The curve rises steeply and then flattens. For breast cancer, about 10 components already keep about 95% of the variance, so 30 features can be cut to 10 with little loss.

G2. What is the dotted line at 0.95? A: A threshold. The first x-value where the curve crosses it is the number of components to keep.

G3. What is a scree plot? A: A plot of the variance explained by each individual component (not cumulative). You look for the elbow where the bars drop off and keep the components before it.

G4. What do PC1 and PC2 on the scatter plot mean? A: PC1 is the direction of maximum variance in the data and PC2 is the next direction, perpendicular to PC1. They are linear combinations of the original features.

G5. The two classes separate in the scatter plot. Why? A: The classes differ strongly along the top directions of variance, so even 2 components carry most of the class information. If the points overlapped heavily, 2 components would be insufficient.

G6. Why does the LDA plot separate classes better than PCA? A: LDA uses the labels and chooses the axis that maximises the gap between class means relative to the spread within each class. PCA ignores the labels.

Viva questions and answers

Q1. What is dimensionality reduction? Why use it? A: Reducing the number of input features while keeping most of the information. It reduces computation, storage, noise and overfitting, helps visualisation, and fights the curse of dimensionality.

Q2. What is the curse of dimensionality? A: As the number of features grows, data becomes sparse, distances lose meaning and models need exponentially more data, so they overfit easily.

Q3. What is PCA? A: Principal Component Analysis is an unsupervised technique that transforms correlated features into a smaller set of uncorrelated variables (principal components), ordered by how much variance they capture.

Q4. How does PCA work step by step? A: (1) Standardise the data. (2) Compute the covariance matrix. (3) Find eigenvalues and eigenvectors. (4) Sort eigenvectors by eigenvalue, largest first. (5) Select the top k. (6) Project the data onto them.

Q5. What are eigenvalues and eigenvectors in PCA? A: An eigenvector is a direction (a principal component). Its eigenvalue says how much variance lies along that direction.

Q6. Why must data be scaled before PCA? A: PCA maximises variance, so features with large units dominate. Scaling gives all features equal weight.

Q7. What is LDA? A: Linear Discriminant Analysis is a supervised technique that finds axes that maximise separation between classes (between-class variance) while minimising spread inside each class (within-class variance).

Q8. Difference between PCA and LDA? A: PCA is unsupervised and maximises total variance. LDA is supervised and maximises class separability. PCA can give up to n_features components; LDA gives at most (number of classes - 1).

Q9. Why is the LDA maximum components c - 1? A: With c classes, the c class means lie in a space of dimension at most c - 1, so there are only c - 1 useful discriminant directions. Binary problem means 1 component.

Q10. Is PCA feature selection? A: No. It is feature extraction: new features are created as combinations of the old ones. Feature selection keeps a subset of original features.

Q11. How do you choose the number of components? A: Keep enough to explain about 95% of the variance (cumulative plot), use the scree-plot elbow, or pick by validation accuracy.

Q12. Did performance fall after reduction? Why? A: Slightly, because some information is discarded. The trade-off is lower computation and simpler model for a small accuracy drop.

Q13. Are principal components interpretable? A: Not easily. Each is a mix of all original features, found through the loadings in `pca.components_`.

Q14. What assumptions does LDA make? A: Features are normally distributed within each class, classes share the same covariance matrix, and the classes are linearly separable.

Q15. Is PCA affected by outliers? A: Yes, because it relies on variance and the covariance matrix, so extreme points can pull the components.

Practical 3: Multiple linear regression (MSE, RMSE, R^2)

Aim: Build a multiple linear regression model on a real-world dataset and assess its predictive capability using MSE, RMSE and R^2.

Dataset: California Housing (predict median house value from 8 features) or Diabetes.

Steps

1. Load data into a DataFrame, check for nulls and look at the correlation heatmap.
2. Split into train and test (80/20).
3. Fit `LinearRegression` on the training data.
4. Print intercept and coefficients.
5. Predict on the test set and compute MSE, RMSE, MAE and R^2.
6. Plot actual vs predicted and a residual plot.

Code

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

data = fetch_california_housing(as_frame=True)
df = data.frame
X, y = df.drop(columns='MedHouseVal'), df['MedHouseVal']

sns.heatmap(df.corr(), annot=True, cmap='coolwarm'); plt.show()

Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression().fit(Xtr, ytr)
print('Intercept:', model.intercept_)
print(pd.Series(model.coef_, index=X.columns))

pred = model.predict(Xte)
mse = mean_squared_error(yte, pred)
print('MSE :', mse)
```

```
print('RMSE:', np.sqrt(mse))
print('MAE :', mean_absolute_error(yte, pred))
print('R2  :', r2_score(yte, pred))

# Actual vs predicted
plt.scatter(yte, pred, alpha=0.4)
plt.plot([yte.min(), yte.max()], [yte.min(), yte.max()], 'r--')
plt.xlabel('Actual'); plt.ylabel('Predicted'); plt.show()

# Residuals
res = yte - pred
plt.scatter(pred, res, alpha=0.4); plt.axhline(0, color='r')
plt.xlabel('Predicted'); plt.ylabel('Residual'); plt.show()
```

Typical result (California Housing): MSE about 0.56, RMSE about 0.75 (that is about $75,000 since the target is in $100,000s), R^2 about 0.58. Meaning: the model explains about 58% of the variance in house prices; it is moderate, not great.

Formulas to remember

```
MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \qquad RMSE = \sqrt{MSE}
```

```
R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}
{\sum (y_i - \bar{y})^2}
```

Graph questions

G1. What does the actual vs predicted plot show? A: Each dot is one test sample. The red dashed line is the perfect-prediction line (y = x). Points close to it mean accurate predictions; a wide scatter means large errors.

G2. Points are scattered widely and form a flat band at the top. Why? A: In California Housing the target is capped at 5.0 ($500,000), so there are many identical actual values, and the linear model under-predicts high prices. It also hints that the relationship is not perfectly linear.

G3. What does the residual plot show and what should a good one look like? A: Residual = actual minus predicted. A good plot is a random cloud around zero with no pattern and constant spread. A curve means non-linearity; a funnel shape means heteroscedasticity (variance changes with the prediction); both mean the linear model is not ideal.

G4. What does the correlation heatmap tell you? A: It shows the pairwise correlation between features (−1 to +1). Strong correlation between the target and a feature (MedInc) means it is a good predictor. Strong correlation between two features signals multicollinearity.

G5. What is the regression line plot for a single feature? A: A scatter of x versus y with the best-fit line through it; the line minimises the sum of squared vertical distances to the points.

Viva questions and answers

Q1. What is linear regression? A: A supervised algorithm that models a continuous target as a linear combination of the input features by fitting a straight line (or hyperplane).

Q2. What is multiple linear regression? Write the equation. A: Regression with two or more features: $y = b0 + b1 \cdot x1 + b2 \cdot x2 + \ldots + bn \cdot xn + \text{error}$, where b0 is the intercept and b1..bn are coefficients.

Q3. How are the coefficients found? A: By Ordinary Least Squares, which minimises the sum of squared residuals. Closed form: $\beta = (X^T X)^{-1} X^T y$. Gradient descent can also be used.

Q4. What does a coefficient mean? A: The expected change in the target for a one-unit increase in that feature, with all other features held constant.

Q5. Explain MSE, RMSE, MAE. A: MSE is the average of squared errors (penalises big errors, units squared). RMSE is its square root (same units as target, easy to interpret). MAE is the average absolute error (less sensitive to outliers).

Q6. Explain R^2. A: The proportion of the variance in the target explained by the model. 1 is perfect, 0 means no better than predicting the mean, and it can be negative for a very bad model.

Q7. What is adjusted R^2 and why use it? A: R^2 never decreases when you add features, even useless ones. Adjusted R^2 penalises extra features, so it only rises when a new feature really helps.

Q8. What are the assumptions of linear regression? A: Linearity, independence of errors, homoscedasticity (constant error variance), normally distributed errors, and no multicollinearity (for multiple regression). Memorise as LINE: Linearity, Independence, Normality, Equal variance.

Q9. What is multicollinearity and how do you detect it? A: When features are highly correlated with each other, making coefficients unstable. Detect it with the correlation matrix or the Variance Inflation Factor (VIF > 5 or 10 is a concern). Fix by dropping features, PCA, or Ridge.

Q10. What is the difference between RMSE and R^2? A: RMSE is an absolute error in the target's units (lower is better). R^2 is a relative, unitless measure of fit (higher is better).

Q11. Why do we use squared errors rather than absolute errors? A: Squared error is smooth and differentiable, giving a unique closed-form solution, and it penalises large errors more.

Q12. Is linear regression sensitive to outliers? A: Yes, because squared errors give outliers a large pull on the line.

Q13. Is scaling needed for linear regression? A: Not for the OLS solution itself, but it helps for gradient descent, regularised models and for comparing coefficient sizes.

Q14. How do you improve a weak R^2? A: Add or engineer features, handle outliers, use polynomial features, use regularisation, or try non-linear models such as Random Forest.

Q15. What is the difference between simple and multiple linear regression? A: Simple has one input feature; multiple has two or more.

Practical 4: Polynomial regression and the bias-variance trade-off

Aim: Apply polynomial regression with different degrees and observe how model complexity affects prediction accuracy (underfitting vs overfitting).

Dataset: Diabetes (use one feature such as BMI for easy plotting) or the Auto MPG dataset (horsepower vs mpg). You can also use a small synthetic curve.

Steps

1. Load data and pick one feature (X) and the target (y).
2. Split into train and test.
3. For each degree (1, 2, 3, 5, 10, 15): create polynomial features, fit linear regression, compute train error and test error.
4. Plot the fitted curves for different degrees on the scatter data.
5. Plot train and test error versus degree.
6. Choose the degree where test error is lowest.

Code

```
import numpy as np, matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import mean_squared_error, r2_score

X, y = load_diabetes(return_X_y=True)
X = X[:, [2]]                                    # BMI only
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=1)

degrees = [1, 2, 3, 5, 8, 12]
train_err, test_err = [], []
```

```
xs = np.linspace(X.min(), X.max(), 300).reshape(-1, 1)

plt.figure(figsize=(12, 7))
for i, d in enumerate(degrees):
    model = make_pipeline(PolynomialFeatures(d), LinearRegression()).fit(Xtr,
ytr)
    train_err.append(mean_squared_error(ytr, model.predict(Xtr)))
    test_err.append(mean_squared_error(yte, model.predict(Xte)))
    plt.subplot(2, 3, i + 1)
    plt.scatter(Xtr, ytr, s=10); plt.plot(xs, model.predict(xs), 'r')
    plt.title(f'Degree {d}  R2(test)={r2_score(yte,
model.predict(Xte)):.2f}')
plt.tight_layout(); plt.show()

plt.plot(degrees, train_err, 'o-', label='Train MSE')
plt.plot(degrees, test_err, 's-', label='Test MSE')
plt.xlabel('Polynomial degree'); plt.ylabel('MSE'); plt.legend(); plt.show()
```

Typical result: Degree 1 has high train and high test error (underfit). Degree 2 to 3 gives the lowest test error (best fit). Degree 8 to 12 has very low train error but rising test error (overfit).

Formula

```
y = b_0 + b_1 x + b_2 x^2 + \dots + b_d x^d
```

Graph questions

G1. What does the curve for degree 1 look like and what does it mean? A: A straight line that misses the curvature of the data. Both train and test errors are high, so it is underfitting (high bias, low variance).

G2. What does the very high degree curve look like? A: A wiggly curve that passes through almost every training point and swings wildly at the edges. Train error is almost zero but test error is large, so it is overfitting (low bias, high variance).

G3. Explain the train/test error versus degree graph. A: Train error keeps falling as the degree rises because the model becomes more flexible. Test error falls first, reaches a minimum (the best degree) and then rises again, giving a U-shape. The gap between the two lines widens with overfitting.

G4. Where is the best model on that graph? A: At the degree with the lowest test error (the bottom of the U), where bias and variance are balanced.

G5. Why does the curve go wild at the ends for high degrees? A: High-degree terms (x^12) grow extremely fast outside the dense data region, so small coefficient changes cause large swings. This is called Runge's phenomenon.

Viva questions and answers

Q1. What is polynomial regression? A: Regression where the relationship between x and y is modelled as an n-th degree polynomial. It is still linear in the coefficients, so linear regression can fit it after creating polynomial features.

Q2. Why is it called a linear model if the curve is non-linear? A: Because it is linear in the parameters (b0, b1, ...). We just transform x into x, x^2, x^3 and fit a linear model on these new features.

Q3. What does `PolynomialFeatures(degree=2)` do? A: For one input x it creates [1, x, x^2]. For two inputs (a, b) it creates [1, a, b, a^2, ab, b^2], including interaction terms.

Q4. When do you use polynomial regression? A: When a scatter plot shows a curved relationship that a straight line cannot capture.

Q5. What is bias? A: Error from wrong or overly simple assumptions in the model. High bias causes underfitting.

Q6. What is variance? A: Error from sensitivity to small changes in training data. High variance causes overfitting, because the model learns the noise.

Q7. Explain the bias-variance trade-off. A: Increasing model complexity lowers bias but raises variance. Total error = bias^2 + variance + irreducible noise. The goal is the sweet spot with minimum total test error.

Q8. How do you detect overfitting? A: Training score is high but test (or validation) score is much lower. Learning curves and cross-validation also show it.

Q9. How do you reduce overfitting? A: Lower the polynomial degree, get more data, use regularisation (Ridge/Lasso), cross-validation, feature selection or early stopping.

Q10. How do you choose the best degree? A: Compare test or cross-validation error for several degrees and choose the lowest, preferring the simplest degree when scores are similar.

Q11. What happens to the number of features as the degree grows? A: It grows quickly (combinatorially) with multiple inputs, which raises computation and the risk of overfitting.

Q12. Should you scale features before polynomial regression? A: Yes, preferably, because x^10 creates huge values and numerical instability.

Q13. Can R^2 on training data alone judge the model? A: No. Training R^2 always rises with degree. Judge on test data.

Q14. What is irreducible error? A: Noise inherent in the data that no model can remove.

Q15. Can polynomial regression extrapolate well? A: No. Outside the training range, high-degree polynomials produce unreliable, extreme predictions.

Practical 5: Regularisation with Ridge and Lasso regression

Aim: Apply Ridge and Lasso regression on a multivariate dataset and observe their influence on overfitting and multicollinearity.

Dataset: Diabetes (10 features, correlated) or California Housing. To show overfitting clearly, you can use polynomial features on top.

Steps

1. Load data, split, and standardise features (regularisation needs scaling).
2. Fit plain Linear Regression as the baseline; record train and test R^2/MSE.
3. Fit Ridge (alpha = 1) and Lasso (alpha = 0.1); record scores.
4. Compare the coefficients of all three models in a table or bar plot.
5. Plot coefficients against alpha (coefficient path) for Ridge and Lasso.
6. Tune alpha with `RidgeCV` and `LassoCV`.

Code

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso, RidgeCV,
LassoCV
from sklearn.metrics import mean_squared_error, r2_score

data = load_diabetes()
X, y, names = data.data, data.target, data.feature_names
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42)
sc = StandardScaler(); Xtr = sc.fit_transform(Xtr); Xte = sc.transform(Xte)

models = {'Linear': LinearRegression(), 'Ridge': Ridge(alpha=10), 'Lasso':
Lasso(alpha=0.5)}
coefs = {}
for n, m in models.items():
    m.fit(Xtr, ytr)
    print(n, 'Train R2 =', round(m.score(Xtr, ytr), 3), 'Test R2 =',
round(m.score(Xte, yte), 3),
          'Test MSE =', round(mean_squared_error(yte, m.predict(Xte)), 1))
    coefs[n] = m.coef_
```

```
coef_df = pd.DataFrame(coefs, index=names)
print(coef_df.round(2))              # Lasso will show some exact zeros
coef_df.plot(kind='bar', figsize=(10, 5)); plt.ylabel('Coefficient');
plt.show()

# Coefficient path
alphas = np.logspace(-3, 2, 50)
ridge_path = [Ridge(alpha=a).fit(Xtr, ytr).coef_ for a in alphas]
lasso_path = [Lasso(alpha=a, max_iter=10000).fit(Xtr, ytr).coef_ for a in
alphas]
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(alphas, ridge_path); ax[0].set_xscale('log');
ax[0].set_title('Ridge path')
ax[1].plot(alphas, lasso_path); ax[1].set_xscale('log');
ax[1].set_title('Lasso path')
for a in ax: a.set_xlabel('alpha'); a.set_ylabel('Coefficients')
plt.show()

print('Best Ridge alpha:', RidgeCV(alphas=alphas).fit(Xtr, ytr).alpha_)
print('Best Lasso alpha:', LassoCV(cv=5).fit(Xtr, ytr).alpha_)
```

Typical result: On the plain 10-feature data, the three models have similar test R^2 (about 0.45 to 0.50). Ridge shrinks coefficients (none exactly zero). Lasso sets some to exactly zero (feature selection). The overfitting gain is clearer if you add polynomial features first: Linear gets high train R^2 and low test R^2, while Ridge and Lasso reduce the gap.

Formulas

```
Ridge:\; \min \sum (y_i - \hat{y}_i)^2 + \alpha \sum_{j} \beta_j^2 \qquad
Lasso:\; \min \sum (y_i - \hat{y}_i)^2 + \alpha \sum_{j} |\beta_j|
```

Graph questions

G1. Explain the coefficient bar plot. A: Each group of bars is one feature; the colours are the models. Linear regression has the largest coefficients (some very large and opposite signs for correlated features like s1, s2). Ridge bars are smaller. Lasso has some bars at zero, so those features are removed.

G2. Explain the Ridge coefficient path. A: The x-axis is alpha (log scale) and the y-axis the coefficient values. At small alpha, the coefficients equal OLS values. As alpha grows, all coefficients shrink smoothly toward zero but never reach it exactly.

G3. Explain the Lasso coefficient path. A: As alpha grows, coefficients hit exactly zero one at a time and stay there. The last features to reach zero are the most important ones.

G4. What is the alpha vs error (validation curve) graph? A: Test or CV error versus alpha. It is U-shaped: tiny alpha gives overfitting, huge alpha gives underfitting (all coefficients near zero), and the minimum is the best alpha.

G5. What does the gap between train R^2 and test R^2 show? A: A large gap means overfitting. Regularisation shrinks the gap.

Viva questions and answers

Q1. What is regularisation and why use it? A: Adding a penalty on the size of coefficients to the loss function. It prevents overfitting, stabilises models with multicollinearity and (Lasso) performs feature selection.

Q2. What is Ridge regression (L2)? A: Linear regression with a penalty of alpha times the sum of squared coefficients. It shrinks coefficients but does not make them zero.

Q3. What is Lasso regression (L1)? A: Linear regression with a penalty of alpha times the sum of absolute coefficients. It can shrink some coefficients exactly to zero, so it selects features.

Q4. Difference between Ridge and Lasso? A: Ridge uses L2 and keeps all features with smaller weights; best when all features matter and are correlated. Lasso uses L1 and removes features; best when many features are irrelevant (sparse models).

Q5. Why does Lasso give exact zeros but Ridge does not? A: The L1 constraint region is a diamond with corners on the axes, so the loss contours often touch at a corner where a coefficient is zero. The L2 region is a circle with no corners, so zeros are rare.

Q6. What does alpha (lambda) control? A: The strength of the penalty. Alpha = 0 gives ordinary linear regression. Larger alpha gives stronger shrinkage, higher bias and lower variance. Too large leads to underfitting.

Q7. How do you choose alpha? A: By cross-validation (`RidgeCV`, `LassoCV`, `GridSearchCV`).

Q8. How does Ridge handle multicollinearity? A: With correlated features, OLS coefficients become huge and unstable. The L2 penalty spreads weight across correlated features and stabilises them.

Q9. How does Lasso behave with correlated features? A: It tends to pick one of them and set the others to zero, which can be arbitrary and unstable.

Q10. What is Elastic Net? A: A combination of L1 and L2 penalties. Useful when features are correlated and you also want sparsity.

Q11. Why scale data before Ridge/Lasso? A: The penalty treats all coefficients equally, so features with large scales would be penalised unfairly. Standardising makes the penalty fair.

Q12. Is the intercept penalised? A: No, only the feature coefficients are.

Q13. What is multicollinearity? A: High correlation between independent variables, which makes coefficient estimates unreliable. Detect with correlation matrix or VIF.

Q14. Effect of regularisation on bias and variance? A: It increases bias slightly and decreases variance, which usually lowers test error.

Q15. When would you not use regularisation? A: When the model already underfits or there are very few features and plenty of data.

Q16. Is Lasso a feature selection method? A: Yes, an embedded one, because selection happens during training.

Practical 6: Logistic regression (confusion matrix, precision, recall, F1)

Aim: Build a binary classifier with logistic regression and evaluate it using the confusion matrix, precision, recall and F1-score.

Dataset: Breast Cancer Wisconsin (malignant vs benign) or Pima Indians Diabetes.

Steps

1. Load data, split with `stratify=y`, standardise.
2. Train `LogisticRegression`.
3. Predict classes and probabilities on the test set.
4. Build the confusion matrix and plot it as a heatmap.
5. Compute accuracy, precision, recall, F1 and print the classification report.
6. (Bonus) Plot the ROC curve and compute AUC.

Code

```
import matplotlib.pyplot as plt, seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (confusion_matrix, accuracy_score,
precision_score,
                             recall_score, f1_score, classification_report,
                             roc_curve, roc_auc_score)

X, y = load_breast_cancer(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
```

```
sc = StandardScaler(); Xtr = sc.fit_transform(Xtr); Xte = sc.transform(Xte)

clf = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
pred = clf.predict(Xte)
prob = clf.predict_proba(Xte)[:, 1]

cm = confusion_matrix(yte, pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Malignant', 'Benign'], yticklabels=['Malignant',
'Benign'])
plt.xlabel('Predicted'); plt.ylabel('Actual'); plt.show()

print('Accuracy :', accuracy_score(yte, pred))
print('Precision:', precision_score(yte, pred))
print('Recall   :', recall_score(yte, pred))
print('F1-score :', f1_score(yte, pred))
print(classification_report(yte, pred))

fpr, tpr, _ = roc_curve(yte, prob)
plt.plot(fpr, tpr, label=f'AUC = {roc_auc_score(yte, prob):.3f}')
plt.plot([0, 1], [0, 1], 'k--'); plt.xlabel('False Positive Rate');
plt.ylabel('True Positive Rate')
plt.legend(); plt.show()
```

Typical result: Accuracy about 97 to 98%, precision about 0.97 to 0.99, recall about 0.98 to 0.99, F1 about 0.98, AUC about 0.99. Example confusion matrix (114 test samples): about 40 TN, 2 FP, 1 FN, 71 TP.

Confusion matrix layout (class 1 = positive)

	Predicted negative (0)	Predicted positive (1)
Actual negative (0)	TN (correct negative)	FP (false alarm, Type I error)
Actual positive (1)	FN (missed case, Type II error)	TP (correct positive)

Formulas

```
Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad Precision = \frac{TP}{TP+FP} \quad
Recall = \frac{TP}{TP+FN} \quad F1 = \frac{2\cdot Precision \cdot Recall}
{Precision + Recall}
```

```
sigmoid(z) = \frac{1}{1+e^{-z}}, \qquad z = b_0 + b_1x_1 + \dots + b_nx_n
```

Graph questions

G1. How do you read the confusion matrix heatmap? A: Rows are actual classes and columns are predicted classes. The diagonal (top-left and bottom-right) shows correct predictions; off-diagonal cells are mistakes. A dark diagonal and light off-diagonal means a good model.

G2. Which cell is the most dangerous in cancer detection? A: A malignant tumour predicted as benign, so the patient is missed. Note: in this dataset 0 = malignant and 1 = benign, so with benign as the positive class this is a False Positive in the sklearn matrix. If you treat malignant as the positive class (more natural for cancer), it is a False Negative and you care most about high recall. Say which class you treat as positive.

G3. What does the ROC curve show? A: The trade-off between True Positive Rate (y-axis) and False Positive Rate (x-axis) at all thresholds. A curve hugging the top-left corner is best. The dashed diagonal is random guessing (AUC = 0.5).

G4. What is AUC? A: Area Under the ROC Curve. 1.0 is perfect, 0.5 is random. It is the probability that the model ranks a random positive above a random negative.

G5. What does the sigmoid curve look like? A: An S-shaped curve from 0 to 1, crossing 0.5 at z = 0. It converts any real number into a probability.

Viva questions and answers

Q1. What is logistic regression? Is it regression or classification? A: Despite the name, it is a classification algorithm. It models the probability that an input belongs to class 1 using the sigmoid function and then applies a threshold (default 0.5).

Q2. Why not use linear regression for classification? A: Linear regression outputs values outside 0 to 1, is sensitive to outliers and its errors are not suited to categories. Logistic regression gives valid probabilities.

Q3. What is the sigmoid function? A: σ(z) = 1 / (1 + e^(-z)). It squashes any input into (0, 1).

Q4. What loss function does logistic regression use? A: Log loss (binary cross-entropy): −[y·log(p) + (1−y)·log(1−p)]. It is minimised by gradient descent or other solvers (lbfgs).

Q5. What is the decision boundary? A: The surface where the predicted probability equals the threshold (0.5), i.e., where z = 0. It is linear in logistic regression.

Q6. What are log-odds? A: log(p / (1−p)) equals the linear combination z. Each coefficient is the change in log-odds for a one-unit change in that feature.

Q7. What is a confusion matrix? A: A table that compares actual and predicted classes: TP, TN, FP and FN counts.

Q8. Define TP, TN, FP, FN. A: TP: predicted positive and actually positive. TN: predicted negative and actually negative. FP: predicted positive but actually negative. FN: predicted negative but actually positive.

Q9. Define precision and recall in plain words. A: Precision: out of everything predicted positive, how many were truly positive. Recall (sensitivity): out of all actual positives, how many did we find.

Q10. What is F1-score and why use it? A: The harmonic mean of precision and recall. It is useful when classes are imbalanced, where accuracy is misleading.

Q11. Why is accuracy not enough? A: With 95% negatives, a model that always predicts negative has 95% accuracy but is useless. Precision, recall and F1 expose this.

Q12. Precision vs recall trade-off: when to prefer which? A: High recall when missing a positive is costly (cancer, fraud). High precision when a false alarm is costly (spam filter blocking real mail).

Q13. How does changing the threshold affect results? A: Lower threshold gives more positives: recall up, precision down. Higher threshold: precision up, recall down.

Q14. What is specificity? A: TN / (TN + FP): the share of actual negatives correctly identified.

Q15. What is the role of the regularisation parameter C? A: C is the inverse of regularisation strength. Small C means stronger regularisation (simpler model); large C means weaker regularisation.

Q16. Can logistic regression do multiclass classification? A: Yes, using One-vs-Rest or the softmax (multinomial) approach.

Q17. What are the assumptions of logistic regression? A: Binary or categorical target, independent observations, little multicollinearity, linear relationship between features and log-odds, and a reasonably large sample.

Q18. Why is `stratify=y` used? A: It keeps the same class proportion in train and test sets, important for imbalanced data.

Practical 7: Support Vector Machines (linear vs non-linear kernels)

Aim: Use SVM classifiers with linear and non-linear kernels on an open-access dataset and compare their performance.

Dataset: Iris (use 2 features for decision-boundary plots) or Breast Cancer / Wine for full-feature comparison.

Steps

1. Load data, split, standardise (SVM needs scaling).
2. Train SVC with kernels: linear, poly, rbf (and optionally sigmoid).
3. Evaluate each with accuracy, confusion matrix and classification report.

4. Compare results in a table.
5. Plot decision boundaries using only 2 features.
6. (Optional) Tune C and gamma with GridSearchCV.

Code

```
import numpy as np, matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

X, y = load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
sc = StandardScaler(); Xtr = sc.fit_transform(Xtr); Xte = sc.transform(Xte)

for k in ['linear', 'poly', 'rbf', 'sigmoid']:
    m = SVC(kernel=k, C=1.0, gamma='scale').fit(Xtr, ytr)
    print(k, 'Accuracy:', round(accuracy_score(yte, m.predict(Xte)), 3))

# Decision boundary with the last 2 features (petal length and width)
X2 = X[:, 2:4]
X2 = StandardScaler().fit_transform(X2)
fig, axes = plt.subplots(1, 2, figsize=(11, 4))
for ax, k in zip(axes, ['linear', 'rbf']):
    m = SVC(kernel=k, C=1, gamma='scale').fit(X2, y)
    xx, yy = np.meshgrid(np.linspace(X2[:,0].min()-1, X2[:,0].max()+1, 300),
                         np.linspace(X2[:,1].min()-1, X2[:,1].max()+1, 300))
    Z = m.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.3)
    ax.scatter(X2[:,0], X2[:,1], c=y, edgecolor='k')
    ax.scatter(m.support_vectors_[:,0], m.support_vectors_[:,1], s=120,
facecolors='none', edgecolors='k')
    ax.set_title(f'{k} kernel')
plt.show()

# Tuning
gs = GridSearchCV(SVC(), {'C': [0.1, 1, 10], 'gamma': [0.01, 0.1, 1],
'kernel': ['rbf']}, cv=5).fit(Xtr, ytr)
print(gs.best_params_, gs.best_score_)
```

Typical result: On Iris both linear and RBF give about 95 to 100% because the data is nearly linearly separable. On non-linear data (make_moons, make_circles) the linear kernel fails (about 85%, or 50% on circles) while RBF reaches about 95 to 100%. If your

dataset is nearly linear, say so; do not claim RBF must win.

Formulas

```
Linear: K(x,z)=x^T z \quad Poly: K(x,z)=(\gamma x^T z + r)^d \quad RBF:
K(x,z)=\exp(-\gamma \lVert x-z \rVert^2)
```

Graph questions

G1. What does the decision boundary plot show? A: The coloured regions are the areas where the model predicts each class; the boundary between colours is the decision boundary. Points are the data, coloured by true class. Points in the wrong colour region are misclassified.

G2. How does the linear boundary differ from the RBF boundary? A: Linear gives straight lines (hyperplanes). RBF gives smooth curved, flexible regions that wrap around clusters.

G3. What are the circled points? A: Support vectors: the training points closest to the boundary (on or inside the margin). Only these points define the hyperplane; other points could be removed without changing it.

G4. What is the margin in the plot? A: The gap between the boundary and the nearest points of each class. SVM maximises this gap.

G5. What would a very high gamma look like? A: Tiny islands around individual points (very wiggly boundary): overfitting. A very low gamma gives an almost straight boundary: underfitting.

G6. What if C is very large? A: Hard margin: the model tries to classify every training point correctly, giving narrow margin and possible overfitting. Small C gives a wider margin and allows some misclassification (more generalisation).

Viva questions and answers

Q1. What is an SVM? A: A supervised algorithm that finds the hyperplane that separates classes with the maximum margin. It works for classification (SVC) and regression (SVR).

Q2. What is a hyperplane? A: A decision surface of dimension n−1 in an n-dimensional space: a line in 2D, a plane in 3D.

Q3. What are support vectors? A: The data points closest to the hyperplane. They alone determine its position and orientation.

Q4. What is the margin and why maximise it? A: The distance between the hyperplane and the nearest points. A larger margin means better generalisation and lower risk of overfitting.

Q5. Hard margin vs soft margin? A: Hard margin allows no misclassification and needs perfectly separable data. Soft margin allows some violations controlled by C, so it works on noisy data.

Q6. What does C control? A: The trade-off between a wide margin and training errors. Large C: fewer errors, narrow margin, overfitting risk. Small C: wide margin, more errors allowed, underfitting risk.

Q7. What is the kernel trick? A: Computing the dot product of data in a higher-dimensional space directly through a kernel function, without actually transforming the data. It lets SVM learn non-linear boundaries cheaply.

Q8. Name common kernels and when to use them. A: Linear: linearly separable or high-dimensional sparse data (text). Polynomial: curved boundaries with a known degree. RBF (Gaussian): the default for general non-linear data. Sigmoid: behaves like a neural network, rarely used.

Q9. What is gamma in the RBF kernel? A: It controls how far the influence of a single training point reaches. High gamma: small influence, complex boundary, overfitting. Low gamma: wide influence, smooth boundary, underfitting.

Q10. Why must features be scaled for SVM? A: SVM relies on distances, and large-range features would dominate the margin and kernel calculation.

Q11. How does SVM do multi-class classification? A: By combining binary classifiers: one-vs-one (scikit-learn SVC default) or one-vs-rest.

Q12. Advantages and disadvantages of SVM? A: Advantages: works well in high dimensions, robust with clear margins, memory-efficient (uses only support vectors). Disadvantages: slow on very large datasets, needs scaling and tuning, no direct probabilities (needs `probability=True`), and poor with heavy noise or overlapping classes.

Q13. Linear SVM vs logistic regression? A: Both are linear classifiers. SVM maximises margin using hinge loss and depends only on support vectors; logistic regression uses log loss and gives probabilities using all points.

Q14. What is the hinge loss? A: $\max(0, 1 - y \cdot f(x))$. It is zero for points correctly classified beyond the margin and grows linearly for violations.

Q15. How do you choose the kernel and parameters? A: Start with linear and RBF, tune C and gamma with GridSearchCV and cross-validation.

Q16. Why did all kernels give similar accuracy here? A: The dataset is almost linearly separable, so a complex kernel adds no benefit. Non-linear kernels help only when the classes are not linearly separable.

Practical 8: K-Means clustering (Elbow and Silhouette)

Aim: Apply K-Means on an open-access dataset and find the optimal number of clusters using the Elbow method or Silhouette score.

Dataset: Iris (ignore the labels), Mall Customers (annual income vs spending score) or Wine.

Steps

1. Load data, drop the label column (clustering is unsupervised), scale features.
2. For k = 1 to 10 run K-Means and store the inertia (WCSS).
3. Plot k vs inertia (Elbow curve) and find the bend.
4. For k = 2 to 10 compute the silhouette score; plot it and pick the highest.
5. Fit the final K-Means with the chosen k.
6. Plot clusters with centroids.

Code

```
import numpy as np, matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

X = load_iris().data
Xs = StandardScaler().fit_transform(X)

inertia, sil = [], []
ks = range(1, 11)
for k in ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(Xs)
    inertia.append(km.inertia_)
    if k > 1:
        sil.append(silhouette_score(Xs, km.labels_))

plt.plot(ks, inertia, 'o-'); plt.xlabel('k'); plt.ylabel('Inertia (WCSS)')
plt.title('Elbow method'); plt.show()

plt.plot(range(2, 11), sil, 's-'); plt.xlabel('k'); plt.ylabel('Silhouette
score')
plt.title('Silhouette analysis'); plt.show()

best_k = 2 + int(np.argmax(sil))
print('Best k by silhouette:', best_k)
```

```
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(Xs)
Xp = PCA(n_components=2).fit_transform(Xs)
plt.scatter(Xp[:,0], Xp[:,1], c=km.labels_, cmap='viridis')
cen = PCA(n_components=2).fit(Xs).transform(km.cluster_centers_)
plt.scatter(cen[:,0], cen[:,1], c='red', marker='X', s=200)
plt.show()
```

Typical result on Iris: The elbow appears at k = 3 (the true number of species), but the silhouette score is highest at k = 2 (about 0.68 for k = 2 vs 0.55 for k = 3 on raw data; about 0.58 vs 0.46 on scaled data), because two species (versicolor and virginica) overlap. Be ready to explain this mismatch; it is a classic viva question. For Mall Customers, both methods give k = 5.

Formulas

```
Inertia = \sum_{i=1}^{n} \lVert x_i - \mu_{c(i)} \rVert^2 \qquad s(i) =
\frac{b(i) - a(i)}{\max(a(i), b(i))}
```

Here a(i) is the mean distance from point i to others in its own cluster, and b(i) is the mean distance to the nearest other cluster.

Graph questions

G1. How do you read the Elbow curve? A: The x-axis is k, the y-axis is inertia (sum of squared distances to centroids). Inertia always falls as k grows. The optimal k is at the elbow, the point where the drop becomes slow and the curve flattens. Adding more clusters beyond it gives little gain.

G2. Why does inertia always decrease with k? A: More centroids means every point is closer to some centroid. With k = n, inertia is zero. So we cannot pick the minimum, only the elbow.

G3. What if the elbow is not clear? A: Use the silhouette score, the gap statistic or domain knowledge.

G4. How do you read the silhouette plot (score vs k)? A: Choose the k with the highest average silhouette score. Values near +1 mean tight, well-separated clusters; near 0 means overlapping clusters; negative means points are probably in the wrong cluster.

G5. What does a silhouette diagram with knife-shaped bars show? A: One bar per cluster, each bar is made of the sorted silhouette values of its points. Wide bars reaching past the average line (red dashed) mean good clusters. Bars that fall below the average line or have negative parts mean poor clusters.

G6. What do the red X marks in the cluster scatter plot mean? A: The centroids, the mean position of every cluster. Each point belongs to the nearest centroid.

G7. Why are the clusters plotted using PCA? A: Data has 4 or more features and cannot be drawn directly; PCA projects it to 2D for visualisation.

Viva questions and answers

Q1. What is clustering? A: Grouping similar data points together without using labels (unsupervised learning).

Q2. How does the K-Means algorithm work? A: (1) Choose k. (2) Initialise k centroids. (3) Assign every point to its nearest centroid. (4) Recompute each centroid as the mean of its points. (5) Repeat steps 3 and 4 until assignments stop changing or max iterations is reached.

Q3. What is inertia / WCSS? A: Within-Cluster Sum of Squares: the total of squared distances between each point and its cluster centroid. Smaller means tighter clusters.

Q4. What is the Elbow method? A: Plot inertia for different k and choose the k at the bend of the curve, after which improvement is marginal.

Q5. What is the silhouette score? A: A measure of how similar a point is to its own cluster compared with other clusters, ranging from −1 to +1. The average over all points rates the clustering.

Q6. Elbow vs silhouette? A: Elbow is visual and can be subjective; it only looks at compactness. Silhouette considers compactness and separation and gives a numeric score to maximise.

Q7. Why is scaling important for K-Means? A: K-Means uses Euclidean distance, so features with larger ranges dominate.

Q8. What is the K-Means++ initialisation? A: A smarter way to choose starting centroids, spreading them far apart. It gives faster convergence and better results than random initialisation (default in scikit-learn).

Q9. What is `n_init`? A: The number of times K-Means runs with different centroid seeds; the best run (lowest inertia) is kept.

Q10. Limitations of K-Means? A: You must choose k in advance; it is sensitive to initialisation and outliers; it assumes spherical, similarly sized clusters; it fails on non-convex shapes (moons, rings); it only works with numeric data.

Q11. Does K-Means always find the global optimum? A: No. It converges to a local optimum, so we run it several times.

Q12. Time complexity? A: $O(n \cdot k \cdot i \cdot d)$, where n = points, k = clusters, i = iterations, d = features. It scales well to large data.

Q13. Difference between K-Means and KNN? A: K-Means is unsupervised clustering and K is the number of clusters. KNN is a supervised classifier and K is the number of neighbours.

Q14. How do you evaluate clustering without labels? A: Silhouette score, Davies-Bouldin index (lower is better), Calinski-Harabasz index (higher is better) and inertia. With labels: Adjusted Rand Index.

Q15. K-Means vs K-Medoids? A: K-Medoids uses actual data points as centres and is more robust to outliers; K-Means uses means.

Q16. Hard vs soft clustering? A: K-Means is hard clustering (each point belongs to exactly one cluster). Gaussian Mixture Models do soft clustering (probabilities).

Practical 9: Density-based clustering with DBSCAN

Aim: Apply DBSCAN on an open-access dataset to identify cluster structure and noise points.

Dataset: `make_moons` (shows DBSCAN's strength over K-Means), Iris, or Mall Customers.

Steps

1. Load or generate data and scale it.
2. Choose `min_samples` (rule of thumb: 2 x number of features, or 4 for 2D data).
3. Find `eps` using the k-distance plot (sorted distance to the k-th nearest neighbour); pick the knee point.
4. Fit DBSCAN; labels of -1 are noise.
5. Count clusters and noise points; compute silhouette score (excluding noise).
6. Plot clusters, with noise points in black.

Code

```
import numpy as np, matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN, KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import silhouette_score

X, _ = make_moons(n_samples=400, noise=0.08, random_state=42)
X = StandardScaler().fit_transform(X)

# k-distance plot to choose eps
min_samples = 5
```

```
d, _ = NearestNeighbors(n_neighbors=min_samples).fit(X).kneighbors(X)
plt.plot(np.sort(d[:, -1])); plt.xlabel('Points sorted by distance');
plt.ylabel(f'{min_samples}-th NN distance')
plt.title('k-distance plot'); plt.show()

db = DBSCAN(eps=0.3, min_samples=min_samples).fit(X)
labels = db.labels_
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
print('Clusters:', n_clusters, '| Noise points:', list(labels).count(-1))

mask = labels != -1
if n_clusters > 1:
    print('Silhouette (no noise):', silhouette_score(X[mask], labels[mask]))

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].scatter(X[:,0], X[:,1], c=KMeans(2, n_init=10).fit_predict(X));
ax[0].set_title('K-Means (fails on moons)')
ax[1].scatter(X[mask,0], X[mask,1], c=labels[mask], cmap='viridis')
ax[1].scatter(X[~mask,0], X[~mask,1], c='k', marker='x');
ax[1].set_title('DBSCAN (black x = noise)')
plt.show()
```

Typical result: DBSCAN finds 2 clusters (the two moons) with a handful of noise points, while K-Means cuts the moons wrongly with a straight line.

Core ideas

Term	Meaning
eps (ε)	Radius of the neighbourhood around a point
min_samples	Minimum points inside eps for a point to be dense
Core point	Has at least min_samples points (including itself) within eps
Border point	Within eps of a core point but has fewer than min_samples neighbours
Noise point	Neither core nor border; label -1

Graph questions

G1. How do you read the k-distance plot? A: Each point's distance to its k-th nearest neighbour is sorted in ascending order. The curve is flat for points in dense regions and then rises sharply for sparse points and outliers. The y-value at the knee (sharp bend) is the best eps.

G2. What happens if eps is too small or too large? A: Too small: most points become noise and many tiny clusters appear. Too large: separate clusters merge into one big cluster.

G3. What do the black crosses mean in the cluster plot? A: Noise points (label -1): points in low-density regions that belong to no cluster.

G4. Why does DBSCAN beat K-Means on the moons plot? A: DBSCAN follows dense regions and can form arbitrarily shaped clusters. K-Means assumes round clusters around centroids, so it splits the moons with a straight boundary.

G5. How many clusters did DBSCAN find and how do you know? A: Count the unique labels excluding -1. The cluster count comes from the data, not from the user.

Viva questions and answers

Q1. What is DBSCAN? A: Density-Based Spatial Clustering of Applications with Noise. It groups points that are closely packed (high density) and marks points in low-density areas as noise.

Q2. What are its two parameters? A: `eps`, the neighbourhood radius, and `min_samples`, the minimum points to form a dense region.

Q3. Explain core, border and noise points. A: Core: at least min_samples points within eps. Border: within eps of a core point but not core itself. Noise: neither.

Q4. Explain the algorithm. A: Pick an unvisited point. If it is a core point, start a new cluster and add all density-reachable points, expanding through other core points. If it is not core, mark it as noise (it may later become a border point). Repeat until all points are visited.

Q5. What is density-reachable and density-connected? A: A point q is density-reachable from p if there is a chain of core points from p to q, each within eps of the next. Two points are density-connected if both are reachable from a common core point.

Q6. DBSCAN vs K-Means? A: DBSCAN needs no number of clusters, finds arbitrary shapes, detects noise and outliers, but is sensitive to eps and struggles with varying densities. K-Means needs k, finds only spherical clusters, assigns every point to a cluster, and is faster on large data.

Q7. How do you choose eps and min_samples? A: min_samples is about 2 x dimensions (at least 3 to 5). eps comes from the k-distance plot knee.

Q8. Is DBSCAN sensitive to scaling? A: Yes, it uses distance, so scale the features first.

Q9. What are the drawbacks? A: Hard to choose eps; fails when clusters have very different densities; poor in high dimensions (distances lose meaning); results can change slightly for border points depending on order.

Q10. What does label -1 mean? A: Noise or outlier.

Q11. Can DBSCAN be used for anomaly detection? A: Yes, the noise points are candidate anomalies.

Q12. What distance metric does it use? A: Euclidean by default; other metrics (Manhattan, cosine) can be set with `metric=`.

Q13. Time complexity? A: O(n log n) with a spatial index such as a KD-tree, O(n^2) without.

Q14. What is HDBSCAN? A: An extension that removes the need for eps and handles clusters with varying density.

Q15. How do you evaluate DBSCAN? A: Silhouette score on non-noise points, Davies-Bouldin index, or Adjusted Rand Index if true labels exist. Also report the noise percentage.

Practical 10: Ensemble learning (Random Forest and Boosting)

Aim: Analyse the performance of Random Forest or Boosting on an open-access dataset and compare it with a baseline model.

Dataset: Breast Cancer or Wine (classification). Baseline: a single Decision Tree or Logistic Regression.

Steps

1. Load data, split with stratification.
2. Train a baseline (Decision Tree).
3. Train Random Forest (bagging) and Gradient Boosting / AdaBoost (boosting).
4. Evaluate all with accuracy, precision, recall, F1 and cross-validation.
5. Plot feature importance from the Random Forest.
6. Compare results in a table and bar chart.

Code

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report

data = load_breast_cancer()
X, y = data.data, data.target
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=42,
stratify=y)
```

```
models = {
    'Decision Tree (baseline)': DecisionTreeClassifier(random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=200,
random_state=42),
    'AdaBoost': AdaBoostClassifier(n_estimators=200, random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=200,
learning_rate=0.1, random_state=42),
}
res = {}
for n, m in models.items():
    m.fit(Xtr, ytr)
    acc = accuracy_score(yte, m.predict(Xte))
    cv = cross_val_score(m, X, y, cv=5).mean()
    res[n] = (acc, cv)
    print(f'{n:28s} Test acc = {acc:.3f}  CV acc = {cv:.3f}')

pd.DataFrame(res, index=['Test', 'CV']).T.plot(kind='bar', ylim=(0.88, 1.0));
plt.ylabel('Accuracy'); plt.show()

rf = models['Random Forest']
imp = pd.Series(rf.feature_importances_,
index=data.feature_names).sort_values()
imp.tail(10).plot(kind='barh'); plt.title('Top 10 feature importances');
plt.show()
```

Typical result: Decision Tree about 91 to 94%, Random Forest about 96 to 97%, Gradient Boosting about 95 to 97%, AdaBoost about 96 to 98%. Ensembles beat the single tree and have more stable cross-validation scores.

Bagging vs boosting at a glance

	Bagging (Random Forest)	Boosting (AdaBoost, Gradient Boosting, XGBoost)
Training	Trees trained in parallel on bootstrap samples	Trees trained one after another, each fixing the previous errors
Main goal	Reduce variance	Reduce bias (and variance)
Combination	Majority vote / average	Weighted sum
Overfitting	Resistant	Can overfit if too many rounds
Speed	Easy to parallelise	Sequential, slower

Graph questions

G1. What does the feature importance bar plot show? A: How much each feature contributes to reducing impurity (Gini) across all trees, normalised to sum to 1. A long bar (for example worst perimeter, worst radius, mean concave points in breast cancer) means that feature is influential.

G2. Can importance tell the direction of the effect? A: No. It shows the strength of influence, not whether the feature raises or lowers the prediction. It can also be biased towards high-cardinality or correlated features.

G3. What does the model comparison bar chart show? A: Accuracy for each model. The ensembles are taller than the baseline tree, meaning combining many learners improves performance.

G4. What does the learning curve (accuracy vs number of trees) show? A: For Random Forest, accuracy rises and then plateaus, so more trees does not overfit. For boosting, test accuracy may peak and then drop, showing overfitting with too many rounds.

G5. What does a confusion matrix of Random Forest show compared with the tree? A: Fewer off-diagonal entries (errors), especially fewer false negatives.

Viva questions and answers

Q1. What is ensemble learning? A: Combining multiple models (weak learners) to get a better, more stable model than any single one.

Q2. Types of ensemble methods? A: Bagging (Random Forest), Boosting (AdaBoost, Gradient Boosting, XGBoost), Stacking and Voting.

Q3. What is bagging? A: Bootstrap Aggregating: train many models on random samples drawn with replacement, then average (regression) or vote (classification). It reduces variance.

Q4. What is Random Forest? A: An ensemble of decision trees built with bagging plus random feature selection at each split, which decorrelates the trees. The final output is the majority vote.

Q5. Why does Random Forest beat a single tree? A: A single deep tree overfits (high variance). Averaging many different trees cancels individual errors and reduces variance.

Q6. What is a bootstrap sample and OOB error? A: A bootstrap sample is drawn with replacement from the training data (about 63% unique rows). The remaining 37% (out-of-bag) rows are used to estimate test error without a separate set (`oob_score=True`).

Q7. What is boosting? A: Sequentially training weak learners where each tries to correct the errors of the previous ones, giving more weight to misclassified examples. It mainly reduces bias.

Q8. AdaBoost vs Gradient Boosting? A: AdaBoost re-weights the misclassified samples and combines learners with weights. Gradient Boosting fits each new tree to the residual errors (the negative gradient of the loss).

Q9. Key hyperparameters of Random Forest? A: `n_estimators`, `max_depth`, `max_features`, `min_samples_split`, `min_samples_leaf`.

Q10. Key hyperparameters of boosting? A: `n_estimators`, `learning_rate` (trade-off: smaller rate needs more trees), `max_depth`, `subsample`.

Q11. Which one is more prone to overfitting? A: Boosting, especially with many rounds and a high learning rate. Random Forest is more robust.

Q12. What is a weak learner? A: A model only slightly better than random guessing, such as a shallow tree (stump).

Q13. Is scaling required for tree ensembles? A: No, trees split on thresholds and are scale-invariant.

Q14. How is feature importance calculated? A: By the total decrease in impurity (Gini or entropy) contributed by splits on that feature, averaged over all trees.

Q15. What is a baseline model and why use it? A: A simple model used as a reference. If a complex model cannot beat it, the added complexity is not justified.

Q16. What are the disadvantages of ensembles? A: Less interpretable, higher computation and memory, longer training time.

Q17. What is a decision tree and what is Gini impurity? A: A tree-shaped model that splits data on feature thresholds. Gini = $1 - \Sigma p_i^2$; it is 0 for a pure node, and the best split reduces it the most.

Q18. What is stacking? A: Training a meta-model on the predictions of several different base models.

Practical 11: Reinforcement learning with Q-Learning (Grid World)

Aim: Evaluate Q-Learning on a simple decision problem (4x4 Grid World), focusing on exploration vs exploitation and the optimal policy.

Setup: A 4x4 grid. Start at (0,0), goal at (3,3) with reward +10, a pit at (1,1) or (2,2) with reward -10, every other step costs -1. Actions: up, down, left, right. The environment is written by hand, so no dataset or library is needed beyond NumPy.

Steps

1. Define states (16 cells), actions (4), rewards and the step function.
2. Create the Q-table (16 x 4) filled with zeros.

3. For each episode: start at the start cell; choose an action with epsilon-greedy; take it; get reward and next state; update Q; repeat until the goal or a step limit.
4. Decay epsilon over episodes (more exploration first, more exploitation later).
5. Record total reward per episode and plot the learning curve.
6. Extract the optimal policy: the best action per state (argmax of Q).

Code

```
import numpy as np, matplotlib.pyplot as plt

N = 4
start, goal, pit = (0, 0), (3, 3), (1, 1)
actions = [(-1, 0), (1, 0), (0, -1), (0, 1)]       # up, down, left, right
arrows = ['^', 'v', '<', '>']

def step(s, a):
    r, c = s[0] + actions[a][0], s[1] + actions[a][1]
    if not (0 <= r < N and 0 <= c < N):
        r, c = s                                    # hit wall: stay
    ns = (r, c)
    if ns == goal: return ns, 10, True
    if ns == pit:  return ns, -10, True
    return ns, -1, False

Q = np.zeros((N, N, 4))
alpha, gamma = 0.1, 0.9
eps, eps_min, decay = 1.0, 0.05, 0.995
episodes = 2000
rewards = []

for ep in range(episodes):
    s, total, done = start, 0, False
    for _ in range(100):
        if np.random.rand() < eps:
            a = np.random.randint(4)               # explore
        else:
            a = np.argmax(Q[s])                    # exploit
        ns, r, done = step(s, a)
        Q[s][a] += alpha * (r + gamma * np.max(Q[ns]) - Q[s][a])   # Bellman
update
        s, total = ns, total + r
        if done: break
    eps = max(eps_min, eps * decay)
    rewards.append(total)
```

```
# Learning curve (moving average)
ma = np.convolve(rewards, np.ones(50) / 50, mode='valid')
plt.plot(ma); plt.xlabel('Episode'); plt.ylabel('Avg reward (50 eps)');
plt.show()

# Optimal policy
for r in range(N):
    print(' '.join('G' if (r, c) == goal else 'X' if (r, c) == pit else
arrows[np.argmax(Q[r, c])] for c in range(N)))
```

Typical result: Early episodes have large negative rewards (random wandering, falling into the pit). After a few hundred episodes the average reward rises to about +4 (10 for the goal minus about 6 step penalties on the shortest path), and the printed policy shows arrows leading from start to goal around the pit.

Formulas

```
Q(s,a) \leftarrow Q(s,a) + \alpha \left[ r + \gamma \max_{a'} Q(s',a') -
Q(s,a) \right]
```

Graph questions

G1. What does the learning curve (reward vs episode) show? A: Learning progress. It starts low and noisy (exploration, many mistakes), rises steadily as Q-values improve, and flattens near the maximum when the policy has converged.

G2. Why is the curve noisy even at the end? A: Because epsilon never reaches zero (epsilon_min = 0.05), so the agent still takes occasional random actions. A moving average smooths it.

G3. How do you read the policy grid? A: Each cell shows the best action (arrow) from that state, i.e. argmax Q(s, a). Following the arrows from start should reach the goal while avoiding the pit.

G4. What does a Q-value heatmap show? A: The maximum Q-value in each state. Values are highest next to the goal and fall with distance (discounting); the pit neighbours are low.

G5. What does an epsilon decay plot show? A: Epsilon falling from 1.0 to 0.05: mostly exploration at first, mostly exploitation at the end.

G6. Compare curves for different epsilon values. A: Too high epsilon: slow learning and low reward because of constant random moves. Too low epsilon: may converge quickly to a poor path because it never explores enough. A decaying epsilon balances both.

Viva questions and answers

Q1. What is reinforcement learning? A: A learning paradigm where an agent interacts with an environment, takes actions, receives rewards and learns a policy that maximises cumulative reward.

Q2. Define agent, environment, state, action, reward, policy. A: Agent: the learner. Environment: the world it acts in. State: the current situation. Action: a choice the agent makes. Reward: scalar feedback. Policy: mapping from states to actions.

Q3. What is Q-Learning? A: A model-free, off-policy, value-based RL algorithm that learns the action-value function Q(s, a): the expected total future reward for taking action a in state s and then acting optimally.

Q4. Explain the Q-learning update rule. A: New Q = old Q + α × (reward + γ × best next Q − old Q). The bracket is the temporal-difference error: the gap between the target and the current estimate.

Q5. What is the learning rate α? A: How much new information overrides old. 0 means no learning; 1 means only the latest experience is used. Typical value 0.1.

Q6. What is the discount factor γ? A: Controls how much future rewards matter. Near 0: short-sighted; near 1: far-sighted. Typical value 0.9 to 0.99.

Q7. What is exploration vs exploitation? A: Exploration: trying new actions to discover better rewards. Exploitation: using current knowledge to pick the best-known action. We need both.

Q8. What is epsilon-greedy? A: With probability ε choose a random action (explore); with probability 1 − ε choose the action with the highest Q (exploit). ε is usually decayed over time.

Q9. Why decay epsilon? A: Early on the Q-table is uninformed, so explore a lot. Later, the policy is good, so exploit more and get higher rewards.

Q10. What is model-free? A: The agent does not know or learn the environment's transition probabilities; it learns directly from experience.

Q11. What is off-policy? A: Q-learning updates using the greedy best next action (max Q), regardless of which action the behaviour policy actually takes. SARSA is on-policy: it uses the action actually taken.

Q12. Q-Learning vs SARSA? A: Q-learning target uses max Q(s′, a′), so it learns the optimal policy and is more optimistic. SARSA uses Q(s′, a′) of the action taken, so it learns a safer policy that accounts for exploration.

Q13. What is the Q-table? A: A table with one row per state and one column per action, storing Q-values.

Q14. What is an optimal policy? A: The policy that maximises expected cumulative reward from every state; derived as $\pi(s)$ = argmax over a of $Q(s, a)$.

Q15. What is the Bellman equation? A: $Q^*(s,a) = E[\, r + \gamma \max Q^*(s', a') \,]$: the value of a state-action equals the immediate reward plus the discounted best value of the next state. Q-learning is a sample-based way to solve it.

Q16. Limitations of tabular Q-learning? A: It does not scale to large or continuous state spaces (the table explodes). Solution: Deep Q-Networks (function approximation).

Q17. What is an episode? A: One complete run from the start state until a terminal state (goal or pit) or step limit.

Q18. What is a Markov Decision Process (MDP)? A: The mathematical framework for RL, defined by states, actions, transition probabilities, rewards and discount factor; the next state depends only on the current state and action (Markov property).

Q19. Does Q-learning always converge? A: Yes in the tabular case, if every state-action pair is visited infinitely often and the learning rate decays properly.

Q20. Why is the step reward −1? A: It penalises long paths, so the agent prefers the shortest route to the goal.

Last-minute cheat sheet

Read this on the way to the exam.

One line per practical

#	Practical	Key idea in one line	Main graph
1	Feature engineering	Impute, encode, scale so the model can use the data	Null heatmap / head()
2	PCA and LDA	PCA keeps max variance (unsupervised); LDA maximises class separation (supervised)	Cumulative variance, 2D scatter
3	Multiple linear regression	$y = b0 + b1x1 + ...$; judged by MSE, RMSE, R^2	Actual vs predicted, residuals
4	Polynomial regression	Higher degree: lower bias, higher variance	Curves by degree, train/test error U-shape
5	Ridge and Lasso	L2 shrinks, L1 shrinks and zeroes coefficients	Coefficient path, bar plot

#	Practical	Key idea in one line	Main graph
6	Logistic regression	Sigmoid gives probability; judged by confusion matrix, P, R, F1	Heatmap, ROC
7	SVM	Maximum-margin hyperplane; kernels for non-linear data	Decision boundary with support vectors
8	K-Means	Centroids and mean updates; choose k by Elbow or Silhouette	Elbow, silhouette, clusters
9	DBSCAN	Density clusters with noise label -1	k-distance plot, clusters with noise
10	Ensembles	Many weak learners beat one; bagging cuts variance, boosting cuts bias	Feature importance, accuracy bars
11	Q-Learning	Learn Q(s,a) by trial and error with epsilon-greedy	Reward vs episode, policy grid

Formulas

Topic	Formula
MSE / RMSE	mean((y − ŷ)²) / √MSE
R^2	1 − SSres / SStot
Accuracy	(TP + TN) / total
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)
F1	2PR / (P + R)
Specificity	TN / (TN + FP)
Standardisation	(x − mean) / std
Min-max	(x − min) / (max − min)
Sigmoid	1 / (1 + e^(−z))
Ridge penalty	α Σ β^2
Lasso penalty	α Σ |β|
Silhouette	(b − a) / max(a, b)

Topic	Formula
Gini	$1 - \Sigma\, p^2$
Q-update	$Q \leftarrow Q + \alpha(r + \gamma \max Q' - Q)$

Frequently confused pairs

Pair	Difference
Bias vs variance	Bias: too simple (underfit). Variance: too sensitive (overfit)
Ridge vs Lasso	L2 keeps all features; L1 selects features
PCA vs LDA	PCA ignores labels; LDA uses them, max c − 1 components
K-Means vs DBSCAN	K-Means needs k, spherical clusters; DBSCAN needs eps, any shape, finds noise
Bagging vs boosting	Parallel, reduces variance vs sequential, reduces bias
Precision vs recall	Precision: of predicted positives, how many are right. Recall: of real positives, how many are found
Label vs one-hot encoding	Ordinal integers vs separate 0/1 columns
Exploration vs exploitation	Try new actions vs use best known action
Q-learning vs SARSA	Off-policy (max) vs on-policy (action taken)
fit vs transform	Learn parameters vs apply them

Viva survival tips

1. Start every answer with a one-line definition, then one example from your dataset.
2. Know your dataset: rows, columns, target, why you chose it, and your final numbers.
3. When you show a graph, say the axes first, then the pattern, then the meaning.
4. If you do not know something, say what you do know about it; do not guess a number.
5. Be ready to explain every line of your code, especially `fit`, `transform`, `train_test_split` and the metrics lines.
6. Your actual numbers may differ from the "typical results" here. That is fine: explain what your numbers mean, not what a textbook says.